

Data Science

Lecture 05: Neural Networks

Prof. Dr. Michael Granitzer

Module 1

Where the Metaphor Comes From

Guiding Question: *A neuron switches a million times slower than a transistor. Why is the brain the metaphor for artificial intelligence at all?*

The Neuron

10^{11}

neurons in the
human brain
(counted)

10^{14}

synapses (estimated)

100k–200k

other neurons a
single neuron's
dendrites connect to

10^{-3} s

the fastest a neuron
switches

A neuron receives signals through its dendrites, integrates them, and fires along its axon when the combined input passes a threshold.

What the Metaphor Buys — and What It Costs

Worth borrowing

Many simple units, massively connected behaviour that emerges from weights rather than from rules learning by adjusting connection strengths.

Not worth claiming

Artificial neurons are **not** models of biological ones. Backpropagation has no established biological counterpart, and no one is simulating a brain.

"Neural network" is a historical name, not a claim about neuroscience. Treat brain analogies in marketing material accordingly.

The Idea Is Old The Success Is Recent

The perceptron dates from 1958, backpropagation from the 1980s. The methods in this lecture are decades old. **What changed in 2012?**

More data

Large labelled datasets that did not previously exist.

More compute

GPUs able to run many passes over those datasets.

Better practice

Activation functions (ReLU), initialisation (Glorot), architectures (ResNet), regularisation (dropout, batch norm), optimisers (RMSProp, ADAM).

What This Implies

The bottleneck was never the idea. It was **data, compute and a set of practical fixes** — which is why "we tried neural networks in 1995 and they did not work" was both true and misleading.

Mini-Exercise

A colleague says: "Deep learning was invented in 2012."

1. What is wrong with that claim?
2. Which of the three columns above was the *binding* constraint before 2012?
3. What would have to change for the next such jump?

Mini-Exercise — Solution

Mini-Exercise — Solution

1. The methods are decades old — perceptron 1958, backpropagation in the 1980s. What arrived in 2012 was the ability to *train them at scale*.

2. Arguably all three, but data and compute were binding: the algorithmic fixes were largely developed once large models became runnable enough to diagnose.

3. Any of them again — a new data regime, cheaper compute, or a fix for a failure mode we currently work around rather than solve.

Takeaway

Neural networks are not new mathematics; they are old mathematics that became trainable. Every "new" technique below exists because a specific failure mode had to be removed.

Module 2

The Perceptron

Guiding Question: *Least squares was nudged by examples it already classified correctly.
What would a learning rule for decisions look like?*

The Model

A perceptron computes a weighted sum of its inputs and applies a **threshold**:

$$h_{\mathbf{w}}(\mathbf{x}) = \text{sign}\left(w_0 + \sum_{j=1}^p w_j x_j\right) = \text{sign}(\mathbf{w}^\top \mathbf{x})$$

Inputs

$x_1, \dots, x_p$, plus a constant
 $x_0 = 1$ carrying the bias w_0

Weights

$w_0, w_1, \dots, w_p$ — **the hypothesis is exactly this vector**

Output

+1 or -1: which side of the boundary the input lies on

This is the same object as *linear regression for classification* from Lecture 04 — a hyperplane and a sign. What is new is how the weights are found.

Weight Adaptation

Perceptron learning rule

For each training example $(\mathbf{x}, y)$ with $y \in \{-1, +1\}$:

$$\Delta w_j = \eta(y - h_{\mathbf{w}}(\mathbf{x}))x_j$$

Weights change **only when the example is misclassified** correctly classified examples leave the hypothesis untouched.

Contrast with least squares, which is nudged by *every* example including the ones it already gets right. This is exactly the defect that made squared loss a poor classification loss.

The Perceptron Convergence Theorem

If the training data is **linearly separable**, the perceptron rule finds a separating hyperplane in a **finite** number of updates — regardless of the starting weights and of the learning rate.

What it promises

Termination with zero training error, guaranteed, when a solution exists.

What it does not promise

Anything at all when the data is **not** separable — the procedure simply never settles. And nothing about *which* separating hyperplane you get, or how it generalises.

Mini-Exercise

A perceptron with $w_0 = -1.5$, $w_1 = 1$, $w_2 = 1$ receives binary inputs.

1. What does it output for $(0, 0)$, $(0, 1)$, $(1, 0)$, $(1, 1)$?
2. Which logical function is this?
3. Which weights would give you OR instead?

Mini-Exercise — Solution

Mini-Exercise — Solution

1. Sums: $-1.5, -0.5, -0.5, +0.5 \rightarrow -1, -1, -1, +1$

2. AND — it fires only when both inputs are 1.

3. $w_0 = -0.5$ with $w_1 = w_2 = 1$: any single 1 now clears the threshold.

The bias w_0 is the threshold, with its sign flipped.

Takeaway

A perceptron is a **hyperplane plus a sign**, learned by correcting only its mistakes. Convergence is guaranteed — but only when a separating hyperplane exists at all.

Three Open Questions

Which solution?

Every separating hyperplane satisfies the theorem. The one you get depends on the initial weights and the order of the examples.

How long?

"Finite" can be very large; the bound depends on the margin between the classes.

And if not separable?

The procedure oscillates forever. Real data usually is not separable.

Zero training error is not the objective — generalisation is. A hyperplane that grazes the training points may sit in the worst possible place for new ones.

Module 3

The Limit of One Layer

Guiding Question: *You built AND and OR with a single perceptron. Why can you not build XOR — and what does that tell us about model families?*

The Question

You built AND and OR with a single perceptron. **Now build XOR:** output 1 when exactly one input is 1.

x_1	x_2	XOR
0	0	0
0	1	1
1	0	1
1	1	0

Try to find weights w_0, w_1, w_2 . Take a minute.

Why It Cannot Work

Plot the four points. The two positive cases sit on **opposite corners** of the square, and so do the two negative ones.

A perceptron can only draw **one straight line**. No single line puts $(0, 1)$ and $(1, 0)$ on one side while keeping $(0, 0)$ and $(1, 1)$ on the other.

XOR is **not linearly separable** — and no amount of training will change that.

The limit is in the **hypothesis space**, not in the learning rule.

The Consequence, Historically

Minsky and Papert's 1969 analysis of exactly this limitation is widely credited with the collapse of neural-network funding for over a decade.

The criticism was mathematically correct and practically overdrawn: the fix — stack a second layer — was already known. What was missing was a way to **train** it.

The Fix: A Hidden Layer

XOR is expressible with two layers — for instance as $(x_1 \vee x_2) \wedge \neg(x_1 \wedge x_2)$: two perceptrons computing OR and NAND, and a third computing AND of their outputs.

The hidden layer builds a **new representation** in which the problem *is* linearly separable. That is what depth buys: not more lines, but better coordinates to draw them in.

A single perceptron is limited by what one hyperplane can separate. Stacking layers removes that limit — which is why the next question is how to train a stack at all.

Why the Threshold Has to Go

The step function

$\text{sign}(\mathbf{w}^\top \mathbf{x})$ has derivative **zero everywhere** it is defined, and no derivative at all at the boundary.

The sigmoid

$$\sigma(z) = \frac{1}{1 + e^{-z}}$$

smooth, bounded, and with a derivative that is non-zero where it matters.

Without a usable gradient there is no way to assign blame to a weight buried in a hidden layer. **Differentiability is what makes depth trainable.**

A Useful Property

$$\sigma'(z) = \sigma(z)(1 - \sigma(z))$$

The derivative is expressible in terms of the output — so a neuron that has already computed its activation can compute its own gradient for free.

Both tails are flat: for large $|z|$ the derivative is nearly zero and learning stalls. This is the **vanishing gradient** problem, and it is why ReLU replaced the sigmoid in deep networks.

Mini-Exercise

1. Compute $\sigma(0)$, and $\sigma'(0)$ using the identity above.
2. A neuron's weighted input is $z = 12$. What is $\sigma'(z)$ roughly, and what does that mean for its weight update?
3. XOR needed a hidden layer. Why did it *also* need this change of activation?

Mini-Exercise — Solution

The sigmoid buys differentiability — the precondition for training depth — at the price of saturating tails where learning stalls.

Mini-Exercise — Solution

1. $\sigma(0) = 0.5$, so $\sigma'(0) = 0.5 \times 0.5 = 0.25$ — the steepest the sigmoid ever gets.

2. $\sigma(12) \approx 0.999994$, so $\sigma' \approx 6 \times 10^{-6}$: the update is effectively zero. The neuron is **saturated** and stops learning.

3. The extra layer makes XOR *expressible* a differentiable activation makes the stack *trainable*. Expressiveness without a gradient is useless.

The sigmoid buys differentiability — the precondition for training depth — at the price of saturating tails where learning stalls.

Module 4

Depth, and How to Train It

***Guiding Question:** A hidden neuron has no target of its own. How much of the final error is it responsible for?*

Structure

Layer

$L_i = \{n_{1,i}, \dots, n_{k,i}\}$ — a set of neurons. L_0 is the input layer.

Fully connected

Every neuron in L_{i-1} feeds every neuron in L_i .

Parameters

$\mathbf{W}_i \in \mathbb{R}^{|L_i| \times |L_{i-1}|}$ — one row of weights per neuron.

The forward pass is then a chain of matrix products with a non-linearity between them:

$$\mathbf{a}_i = \sigma(\mathbf{W}_i \mathbf{a}_{i-1}), \quad \mathbf{a}_0 = \mathbf{x}$$

Why the Non-Linearity Is Not Optional

Without σ , a stack of layers is $\mathbf{W}_3\mathbf{W}_2\mathbf{W}_1\mathbf{x}$ — a **single matrix**, hence a single linear model. Depth without non-linearity buys nothing at all.

Each layer transforms the representation; the final layer draws a hyperplane in the coordinates the earlier layers built. This is XOR's fix, generalised.

Mini-Exercise

A network has 4 inputs, one hidden layer of 5 neurons, and 3 outputs, fully connected, with a bias per neuron.

How many parameters does it have?

Mini-Exercise — Solution

A multilayer network is matrix products separated by non-linearities. **The non-linearity is what makes depth mean anything**, and each hidden layer is a learned change of coordinates.

Mini-Exercise — Solution

$$\underbrace{(4 \times 5) + 5}_{\text{hidden layer}} + \underbrace{(5 \times 3) + 3}_{\text{output layer}} = 25 + 18 = 43$$

Small networks have many parameters; the counts grow as the **product** of adjacent layer sizes, which is why parameter counts explode with width.

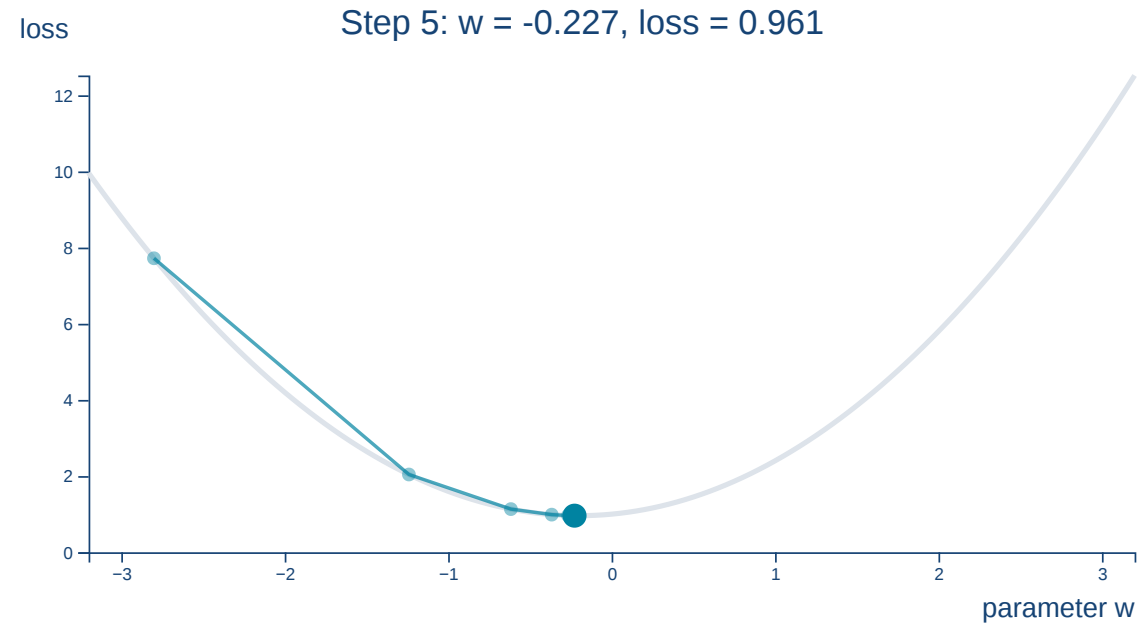
A multilayer network is matrix products separated by non-linearities. **The non-linearity is what makes depth mean anything**, and each hidden layer is a learned change of coordinates.

The Error Surface

For weights $\mathbf{w}$ and training set D , the squared error

$$E(\mathbf{w}) = \frac{1}{2} \sum_{(\mathbf{x}, y) \in D} (y - h_{\mathbf{w}}(\mathbf{x}))^2$$

is a **surface over the weight space**. Training means walking downhill on it.



Two Ways to Descend

Batch gradient descent

Accumulate the gradient over **all** examples, then take one step.

Stable, expensive per step, and it needs the whole dataset in memory.

Incremental (stochastic) gradient descent

Update after **each** example.

Noisier, far cheaper per step, and the noise can help escape poor regions.

The **delta rule**, $\Delta w_j = \eta(y - \mathbf{w}^\top \mathbf{x})x_j$, is incremental gradient descent on the squared error — the LMS rule from Lecture 04 under a new name.

Mini-Exercise

You are training on 1,000,000 examples and each pass over the data takes an hour.

1. How many weight updates does **batch** gradient descent make per hour?
2. How many does **incremental** make?
3. Which would you start with, and what is the risk?

Mini-Exercise — Solution

Mini-Exercise — Solution

1. One.

2. One million — each on a noisy estimate of the true gradient.

3. Incremental (or mini-batch): progress per unit of computation is vastly higher. The risk is instability — the noisy steps can diverge if η is too large.

Takeaway

Training is descent on an error surface. **Batch** steps are accurate and rare; **incremental** steps are noisy and frequent — and on real data, frequency wins.

The Credit Assignment Problem

The output layer's error is measurable: we know the target. But a hidden neuron has **no target of its own**.

How much of the final error is this hidden weight responsible for?

The Answer: Blame Flows Backwards

Output layer

The error is computed directly against the target $y(\mathbf{x})$.

Hidden layers

The error of a neuron is the **weighted sum of the errors of the neurons it feeds** — each scaled by the weight through which it contributed.

A neuron that contributed strongly to a wrong answer receives a large share of the blame. That is the whole idea; the calculus is the chain rule applied layer by layer.

The Algorithm

Backpropagation (incremental form)

repeat **for each** $(\mathbf{x}, y) \in D$: **forward**: compute activations layer by layer up to the output **backward**: compute the output error, then propagate it to each earlier layer
 update: $w \leftarrow w - \eta \cdot (\text{error at the target neuron}) \cdot (\text{activation at the source neuron})$ **until** convergence **or** $t > t_{\max}$

Both passes cost about the same as one forward pass. Training is therefore roughly three times the cost of inference — the fact behind every training-cost estimate you read.

Mini-Exercise

A hidden neuron feeds two output neurons through weights 0.8 and -0.2 . On one example the output errors are $\delta_1 = 0.5$ and $\delta_2 = 1.0$.

1. Which output does the hidden neuron get more blame from?
2. Compute its propagated error (before the activation derivative).
3. What happens to the update if this neuron's activation was near 0?

Mini-Exercise — Solution

Mini-Exercise — Solution

1. Output 1: the weight is four times larger in magnitude, so more of its error routes back here.

2. $0.8 \times 0.5 + (-0.2) \times 1.0 = 0.2$ — the negative weight partly cancels.

3. The update is proportional to the source activation, so a near-zero activation produces a near-zero update: that weight barely learns.

Takeaway

Backpropagation solves **credit assignment**: error flows backwards, weighted by the connections that carried the signal forward. It is the chain rule, applied once per layer — no more and no less.

Module 5

Making Training Work

Guiding Question: *The network fits the training data perfectly and fails on new data. Which of your assumptions was wrong?*

Two Failure Modes

Local minima

A point that is lowest in its neighbourhood but not globally: $E(\mathbf{w}_{\text{opt}}) \leq E(\mathbf{w}_{\text{local}})$.

In practice, large networks appear to have **many** local minima, most of them close in value to the best one — so this matters less than intuition suggests.

Flat regions

Near-zero gradients produce near-zero steps. Training appears to have converged when it has merely stalled — the sigmoid's saturated tails are the usual cause.

Momentum

$$\Delta \mathbf{w}_t = -\eta \nabla E(\mathbf{w}_t) + \alpha \Delta \mathbf{w}_{t-1}$$

Each step keeps a fraction α of the previous one: consistent directions accumulate speed, oscillating directions cancel.

Momentum helps through flat regions and damps zig-zagging in narrow valleys — but the learning rate η remains critical for convergence.

Adaptive Learning Rates

AdaGrad

One learning rate **per parameter**, scaled by the accumulated sum of squared gradients.

Parameters with large gradients slow down quickly rarely-updated parameters keep a large step size.

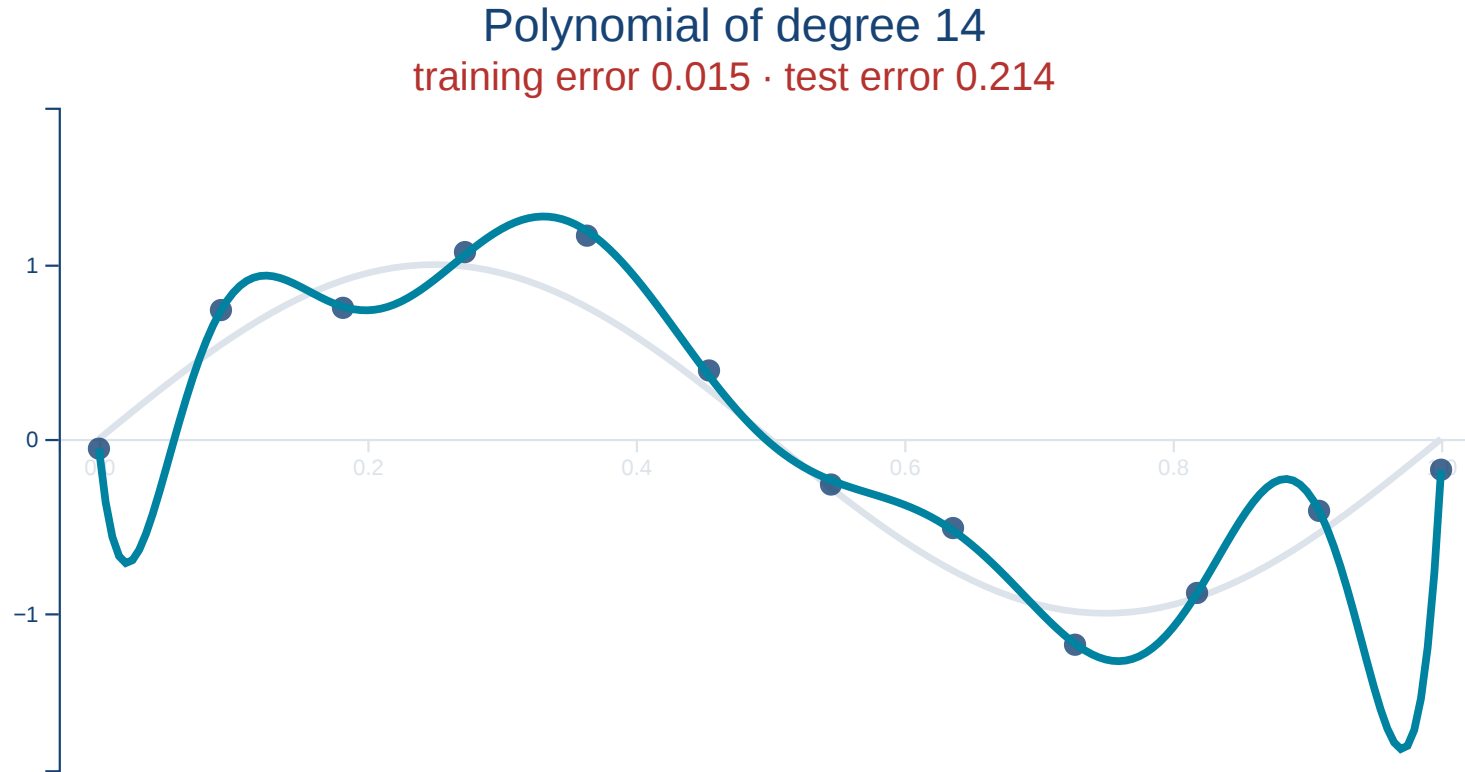
And its successors

RMSProp replaces the ever-growing sum with a moving average **ADAM** combines that with momentum, and is the common default today.

The obstacles are flat regions and badly scaled parameters, not mythical local minima. **Momentum** adds inertia, **adaptive methods** give each parameter its own step size — and both exist to remove a specific, diagnosable failure.

Capacity Cuts Both Ways

A network with enough neurons can fit **any** training set — including its noise. The decision surface becomes intricate where the data is sparse.



Regularization

Add a penalty on the weights to the loss:

$$E_{\text{reg}}(\mathbf{w}) = E(\mathbf{w}) + \lambda \|\mathbf{w}\|$$

where λ controls how much the penalty counts.

$$L_2 \text{ — } \|\mathbf{w}\|_2$$

Penalises **peaky** weight vectors and prefers diffuse ones: every input to a neuron gets used a little. The default choice.

$$L_1 \text{ — } \|\mathbf{w}\|_1$$

Produces **sparse** weights (many exact zeros): a neuron uses only its most important inputs, so it acts as a feature selector.

A regulariser is a way of writing **background knowledge** into the objective — "prefer simple explanations", or "expect most inputs to be irrelevant". It is the inductive bias from

Dropout

During training, each neuron's output is **silenced with probability** $1 - p$. The network cannot rely on any single unit, so it must spread the representation across many. Because training silences a fraction of the units, a layer's total output is smaller during training than at inference. The surviving activations are therefore scaled by $\frac{1}{p}$ to compensate.

Dropout is active during **training only**. Forgetting to disable it at inference time is a classic bug.

Mini-Exercise

Your network reaches 99% training accuracy and 71% validation accuracy.

1. What is the diagnosis?
2. Name three different interventions.
3. Which one would you try first, and why?

Mini-Exercise — Solution

Mini-Exercise — Solution

1. Overfitting — a 28-point gap between training and validation.

2. More data stronger regularisation (λ , dropout) a smaller network early stopping.

3. More data if it is obtainable — it reduces variance without adding assumptions. Otherwise regularisation, because it is the cheapest to try and its effect is directly measurable.

Takeaway

Capacity that can fit the signal can also fit the noise. **Regularisation encodes what you believe about the unseen data** — and the gap between training and validation error is how you find out whether you believed correctly.

Module 6

Learning the Representation

Guiding Question: *In Lecture 04 we chose the features by hand. Can a network choose them for us — and how would we know if it did?*

Learning to Reproduce the Input

An **autoencoder** is trained so that its output reproduces its input, through a hidden layer that is **narrower** than the input.

Encoder

maps the input to the narrow hidden layer

Bottleneck

a compressed code — the representation being learned

Decoder

reconstructs the input from that code

No labels are required: the target *is* the input.

What Makes It Work

The bottleneck does the work

A narrow hidden layer can be read as a **noisy compression** of the input: the network must discard whatever it can afford to lose.

Too wide and it cheats

If the hidden layer is large enough, the network simply learns the **identity function** and has represented nothing.

Denoising autoencoders

Corrupt the input, then require reconstruction of the *clean* version — the identity function is no longer a solution.

Weight sharing

Tie the decoder to the encoder,
 $\mathbf{W}_{1,2} = \mathbf{W}$
 $\{0,1\}^{\text{top}}$, halving the parameters.

For binary inputs, cross-entropy replaces squared error as the reconstruction loss — the

Mini-Exercise

You train an autoencoder with a 784-dimensional input and a 784-neuron hidden layer. Reconstruction error goes to almost zero.

1. Is this a good result?
2. What has the network probably learned?
3. Give two ways to fix the setup.

Mini-Exercise — Solution

An autoencoder learns a representation by being **forced to lose information**. Remove the constraint and it learns nothing — a reminder that the objective, not the architecture, decides what a model learns.

Mini-Exercise — Solution

1. No — near-zero error here is a warning sign, not a success.

2. The identity function: copy the input across. No compression, no representation.

3. Narrow the bottleneck or keep the width and corrupt the input (denoising), so copying no longer solves the task.

An autoencoder learns a representation by being **forced to lose information**. Remove the constraint and it learns nothing — a reminder that the objective, not the architecture, decides what a model learns.